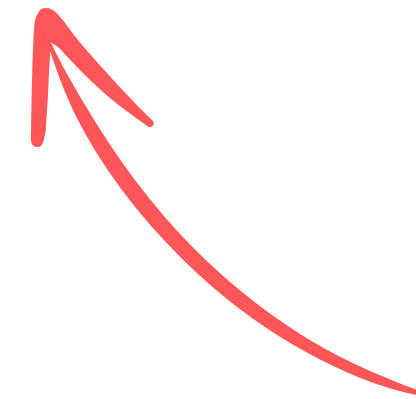


Do Java 1.5

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
string = list.get(0);
```

До Java 1.5

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
string = list.get(0);
```



get возвращает тип **Object**

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
string = (String)list.get(0);
```

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```



```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```



ClassCastException

**Начиная с Java 1.5
были добавлены
Generics**

```
ArrayList<String> strList = new ArrayList<>();  
ArrayList<Integer> intList = new ArrayList<>();  
String string = "Hello World";
```

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```

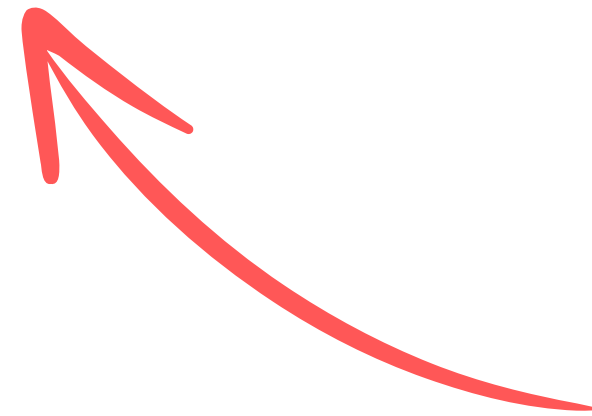
ClassCastException


```
ArrayList<String> strList = new ArrayList<>();  
ArrayList<Integer> intList = new ArrayList<>();  
String string = "Hello World";  
strList.add(5);
```

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```

ClassCastException

```
ArrayList<String> strList = new ArrayList<>();  
ArrayList<Integer> intList = new ArrayList<>();  
String string = "Hello World";  
strList.add(5);
```



Компилятор не пропустит

```
ArrayList list = new ArrayList();  
String string = "Hello World";  
list.add(string);  
list.add(5);  
string = (String)list.get(0);
```

ClassCastException

После компиляции произойдёт
СТИРАНИЕ ТИПОВ

После компиляции произойдёт
СТИРАНИЕ ТИПОВ

В коде **дженериков** не будет,
будут снова **сырые типы**
с расставлением приведения
где необходимо

дженерики существуют
только для компилятора
на этапе компиляции


```
class TestClass <T>{  
    private void handle (T argument){  
    }  
  
    private void handle (Object obj){  
    }  
}
```

```
class TestClass <T>{  
    private void handle (T argument){  
    }  
  
    private void handle (Object obj){  
    }  
}
```

```
class TestClass <T>{  
    private void handle (T argument){  
    }  
    private void handle (Object obj){  
    }  
}
```

both methods have same erasure


```
class TestClass <T extends Comparable<T>>
{
    private void handle (T argument){
    }

    private void handle (Object obj){
    }
}
```

<T> - любой класс

- ✓   Object (java.lang)
- ✓   Context (android.content)
 - ✓   ContextWrapper (android.content)
 - ✓   ContextThemeWrapper (android.view)
 - ✓   Activity (android.app)
 - ✓   ComponentActivity (androidx.core.app)
 - ✓   ComponentActivity (androidx.activity)
 - ✓   FragmentActivity (androidx.fragment.app)
 - ✓   AppCompatActivity (androidx.appcompat.app)
 - *   ModernAddNotificationActivity (ru.salon.screens.settings.modern)

<T extends Activity> - Класс Activity и классы которые ниже (правее) по иерархии наследования

- Object (java.lang)
 - Context (android.content)
 - ContextWrapper (android.content)
 - ContextThemeWrapper (android.view)
 - Activity (android.app)
 - ComponentActivity (androidx.core.app)
 - ComponentActivity (androidx.activity)
 - FragmentActivity (androidx.fragment.app)
 - AppCompatActivity (androidx.appcompat.app)
 - * ModernAddNotificationActivity (ru.salon.screens.settings.modern)

```
class TestClass <T>{  
    private void handle (T argument){  
        argument. ?  
    }  
}
```

```
class TestClass <T extends Comparable<T>{  
    private void handle (T argument){  
    }  
}
```

```
class TestClass <T>{  
    private void handle (T argument){  
        argument.  
            equals()  
            getClass()  
            hashCode()  
            toString()  
            wait()  
            notify()  
            notifyAll()  
    }  
}
```

```
class TestClass <T extends Comparable<T>{  
    private void handle (T argument){  
    }  
}
```



```
class TestClass <T>{  
    private void handle (T argument){  
        argument.  
            equals  
            compareTo  
            getClass  
            hashCode  
            toString  
            wait  
            notify  
            notifyAll  
    }  
}
```

```
class TestClass <T extends Comparable<T>{  
    private void handle (T argument){  
        arguments. ?  
    }  
}
```



```
class TestClass <T>{  
    private void handle (T argument){  
        argument.  
            equals()  
            getClass()  
            hashCode()  
            toString()  
            wait()  
            notify()  
            notifyAll()  
    }  
}
```

```
class TestClass <T extends Comparable<T>{  
    private void handle (T argument){  
        arguments.  
            equals()  
            compareTo()  
            getClass()  
            hashCode()  
            toString()  
            wait()  
            notify()  
            notifyAll()  
    }  
}
```

ИНВАРИАНТНОСТЬ

КОВАРИАНТНОСТЬ

КОНТРВАРИАНТНОСТЬ


```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

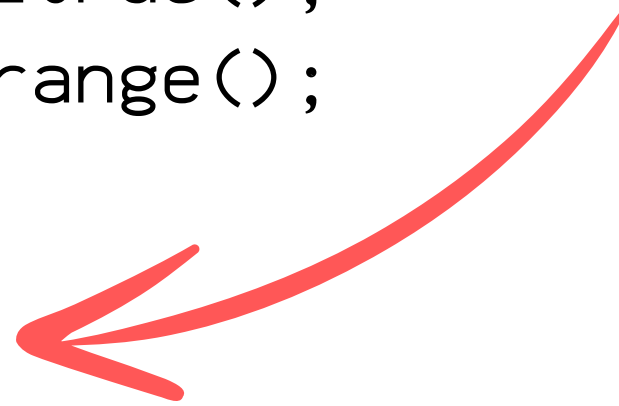
```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```



дочерний класс содержит
в себе все поля и методы
базового класса
и может быть использован
в любом месте где
нужен родительский класс


```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```



```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```



**мы объявили переменную в
середине иерархии классов
в эту переменную можно
записать классы потомки,
из этой переменной можно
записать в родительский класс**

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```



```
ArrayList<Citrus> citrusList = new ArrayList<>();
```



```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```



```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);
```

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);  
orange = citrusList.add(1);
```



```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);  
orange = citrusList.add(1);
```



```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);  
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}

class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
```



```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```



```
ArrayList<Orange> orangeList = new ArrayList<>();
```

```
citrusList = orangeList; ?
```



```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);  
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
```

```
citrusList = orangeList;
```

```
class Fruit{  
    int weight;  
}
```

```
class Citrus extends Fruit{  
    int color;  
}
```

```
class Orange extends Citrus{  
    int size;  
}
```

```
Fruit fruit = new Fruit();  
Citrus citrus = new Citrus();  
Orange orange = new Orange();
```

```
citrus = orange;  
fruit = citrus;  
fruit = orange;
```

```
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();  
fruit = citrusList.add(2);  
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
```

```
citrusList = orangeList;
```



если мы так присвоим, то мы сможем
через citrusList вставить экземпляр citrus,
и изначально коллекции orange
появится citrus. Тогда, например, при
дальнейшей работе с полями oranges
citrus, не имея поля size ломает
программу

нарушается типобезопасность

в Java обобщенные
классы

ИНВАРИАНТЫ


```

class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}

class Orange extends Citrus{
    int size;
}

```

```

Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();

```

```

citrus = orange;
fruit = citrus;
fruit = orange;

```

```

citrus = fruit;
citrus = fruit;

```

```

ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);

```

```

ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;

```

```

static int knowWeight(ArrayList<Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    return weight;
}

```

knowWeight(orangeList); ?

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    return weight;
}
```

```
knowWeight(orangeList);
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++){
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}

class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++){
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
static void addOrange(ArrayList<Orange> orange){
    for(int i = 0; i < 10, i++)
        orange.add(new Orange());
    } return weight;
}
```



```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
static void addOrange(ArrayList<Orange> orange){
    for(int i = 0; i < 10, i++)
        orange.add(new Orange());
    } return weight;
}
```

```
addOrange(citrusList);
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
static void addOrange(ArrayList<Orange> orange){
    for(int i = 0; i < 10, i++)
        orange.add(new Orange());
    } return weight;
}
```

```
addOrange(citrusList);
```

```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    return weight;
}
```

```
knowWeight(orangeList);
```

```
static void addOrange(ArrayList<? super Orange> orange){
    for(int i = 0; i < 10, i++)
        orange.add(new Orange());
    return weight;
}
```

```
addOrange(citrusList);
```



```
class Fruit{
    int weight;
}

class Citrus extends Fruit{
    int color;
}
```

```
class Orange extends Citrus{
    int size;
}
```

```
Fruit fruit = new Fruit();
Citrus citrus = new Citrus();
Orange orange = new Orange();
```

```
citrus = orange;
fruit = citrus;
fruit = orange;
```

```
citrus = fruit;
citrus = fruit;
```

```
ArrayList<Citrus> citrusList = new ArrayList<>();
fruit = citrusList.add(2);
orange = citrusList.add(1);
```

```
ArrayList<Orange> orangeList = new ArrayList<>();
citrusList = orangeList;
```

```
static int knowWeight(ArrayList<? extends Citrus> orange){
    int weight = 0;
    for(int i = 0; i < citrus.size(), i++)
        weight += citrus.get(i).weight;
    } return weight;
}
```

```
knowWeight(orangeList);
```

```
static void addOrange(ArrayList<? super Orange> orange){
    for(int i = 0; i < 10, i++)
        orange.add(new Orange());
    } return weight;
}
```

```
addOrange(citrusList);
```

| Тип | Что можно
присвоить
= | Что можно
читать
get() | Что можно
добавлять
add() |
|--|---|------------------------------|---------------------------------|
| Инвариантный
<code>List<Type></code> | только
<code>List<Type></code> | Type
и предки Type | Type
и наследники Type |
| Ковариантный
<code>List<? extends Type></code> | <code>List<Type></code>
и List
наследников Type | Type
и предки Type | Ничего |
| Контрвариантный
<code>List<? super Type></code> | <code>List<Type></code>
и List
предков Type | Object | Type
и наследники Type |